

Sheet 01 - Regression & Classification

Introduction to Deep Learning - Summer Semester 2026

Ulf Krumnack & Robin Rawiel - Universität Osnabrück

Due: April 19, 2026

Task 1: Theory Questions [6 points]

1.1 Loss Functions [2 points]

1. Define the **Mean Squared Error (MSE)** loss function. For a dataset $\{(x_i, y_i)\}_{i=1}^N$ and a model f_θ , write down the MSE loss $\mathcal{L}(\theta)$.
2. Write down the **binary cross-entropy** loss for a single sample (x, y) where $y \in \{0, 1\}$ and $\hat{y} = \sigma(\theta^\top x)$ is the predicted probability.
3. Consider the **Mean Absolute Error (MAE)** loss $\mathcal{L} = \frac{1}{N} \sum_i |y_i - f_\theta(x_i)|$. Compare it to MSE:
 - Which loss is more sensitive to outliers? Justify your answer.
 - Which loss is differentiable everywhere? Why does this matter for gradient-based optimization?

Hint

The MAE penalises errors *linearly* — each residual $|y_i - f_\theta(x_i)|$ contributes proportionally to its magnitude — whereas MSE penalises errors *quadratically*. Think about what squaring does to large vs. small residuals.

1.2 Maximum Likelihood Estimation [2 points]

1. Assume the data-generating process is $y = f_\theta(x) + \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Show that maximizing the log-likelihood of the data under this model is equivalent to minimizing the squared error loss.
2. Now assume ε follows a **Laplace distribution** $p(\varepsilon) = \frac{1}{2b} \exp(-\frac{|\varepsilon|}{b})$. Which loss function does maximum likelihood estimation correspond to in this case?

Hint

Follow the same log-likelihood derivation strategy as in 1.2.1: write down $p(y_i | x_i, \theta)$ under the Laplace noise assumption, take the log, and identify which terms depend on θ . Compare the result to the MAE formula from 1.1.3.

1.3 Softmax & Evaluation [2 points]

1. Write down the **softmax function** mapping a logit vector $z \in \mathbb{R}^K$ to a probability vector. Show that $\text{softmax}(z) = \text{softmax}(z - c)$ for any constant c and explain why this is useful for numerical implementation.
2. Define **accuracy**, **precision**, and **recall** using the entries of the confusion matrix (TP, TN, FP, FN). When is accuracy a misleading metric?

Task 2: Linear Regression with PyTorch [6 points]

Learning objectives:

- Get familiar with PyTorch tensors and basic linear algebra operations
- Implement the closed-form (normal equation) solution for linear regression
- Visualize data and the fitted regression line

2.1 Generate Synthetic Data [1 point]

Generate $N = 100$ data points from the model $y = 3x + 2 + \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, 1)$ using PyTorch tensors. Plot the data.

```
import torch
import matplotlib.pyplot as plt

torch.manual_seed(42)

# TODO: Your solution here
```

2.2 Closed-Form Solution [3 points]

Implement the **normal equation** $\theta = (X^T X)^{-1} X^T y$ using PyTorch tensor operations to find the optimal parameters $[w, b]$.

```
def closed_form_solution(x, y):
    """Compute optimal [slope, intercept] via the normal equation.

    Args:
        x: Input features, shape (N,)
        y: Targets, shape (N,)

    Returns:
        theta: Tensor [slope, intercept]
    """
    # TODO: Your solution here

# TODO: Your solution here
```

2.3 Visualize the Fit [2 points]

Plot the data points together with the fitted regression line. Also plot the true line $y = 3x + 2$ for comparison.

```
# TODO: Your solution here
```

Task 3: Logistic Regression with PyTorch [8 points]

Learning objectives:

- Implement sigmoid and binary cross-entropy using PyTorch tensors
- Train a binary classifier using gradient descent
- Visualize decision boundaries

3.1 Activation Functions [1 point]

Implement numerically stable versions of sigmoid and softmax using PyTorch.

💡 Hint: Numerical stability of sigmoid

The naïve formula $\sigma(z) = \frac{1}{1+e^{-z}}$ can overflow when z is a large negative number (because e^{-z} explodes). An equivalent form $\sigma(z) = \frac{e^z}{1+e^z}$ is safe for $z < 0$ but overflows for large positive z . Use `torch.where` to pick the appropriate formula depending on the sign of z .

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

def sigmoid(z):
    """Numerically stable sigmoid function.

    Args:
        z: Input tensor.

    Returns:
        Tensor of same shape with values in (0, 1).
    """
    # TODO: Your solution here

def softmax(z):
    """Numerically stable softmax. z has shape (N, K).

    Args:
        z: Input tensor, shape (N, K).

    Returns:
        Tensor of same shape, rows sum to 1.
    """
    # TODO: Your solution here
```

3.2 Data and Loss [2 points]

Generate the moons dataset and implement binary cross-entropy.

```
# Generate data
X_np, y_np = make_moons(n_samples=300, noise=0.2, random_state=42)
X_train_np, X_test_np, y_train_np, y_test_np = train_test_split(
    X_np, y_np, test_size=0.2, random_state=42
)

# Convert to PyTorch tensors
X_train = torch.tensor(X_train_np, dtype=torch.float32)
X_test = torch.tensor(X_test_np, dtype=torch.float32)
y_train = torch.tensor(y_train_np, dtype=torch.float32)
y_test = torch.tensor(y_test_np, dtype=torch.float32)

plt.scatter(X_train_np[:, 0], X_train_np[:, 1], c=y_train_np, cmap="bwr", alpha=0.6)
plt.title("Moons Dataset (Train)")
plt.xlabel("$x_1$")
plt.ylabel("$x_2$")
```

```
plt.show()

def binary_cross_entropy(y_true, y_pred):
    """Compute the mean binary cross-entropy loss.

    Args:
        y_true: True labels, shape (N,), values in {0, 1}.
        y_pred: Predicted probabilities, shape (N,), values in (0, 1).

    Returns:
        Scalar loss.
    """
    # TODO: Your solution here
```

3.3 Training [2 points]

Implement logistic regression training with gradient descent.

```
def logistic_regression_train(X_train, y_train, lr=0.1, n_iterations=1000):
    """Train binary logistic regression using gradient descent.

    Args:
        X_train: Training features, shape (N, D).
        y_train: Training labels, shape (N,).
        lr: Learning rate.
        n_iterations: Number of gradient descent steps.

    Returns:
        w: Weights, shape (D,).
        b: Bias (scalar tensor).
        losses: List of loss values per iteration.
    """
    # TODO: Your solution here
```

```
# TODO: Your solution here
```

3.4 Evaluate and Visualize [2 points]

Compute accuracy on train and test sets. Plot the decision boundary.

```
def predict(X, w, b, threshold=0.5):
    """Predict class labels."""
    # TODO: Your solution here

def plot_decision_boundary(X, y, w, b):
    """Plot data points and the linear decision boundary."""
    # TODO: Your solution here
```

```
# TODO: Your solution here
```

3.5 Reflection [1 point]

1. Why can logistic regression not perfectly separate the moons dataset?
2. What would you change to achieve a better fit?